Fluent BioSciences

PIPseeker™ v2.1

User Guide

Doc ID: **FB0002787**

Revision: **3.1**

State: **Release**

Table of Contents

Introduction 3
System Requirements 4
Pipeline Overview 4
FASTQ Processing 5
Structure of the FASTQ input files 5
Barcode Matching 5
R2 Trimming 6
Downsampling 6
Outputs 6
Mapping 6
Parsing the Mapping File 7
Transcriptome vs. Genome Alignment 7
Transcript Counting 7
Multimapped Reads 7
Cell Calling 8
Outputs 9
Manual Selection of Number of Cells 10
Clustering 10
Preprocessing Steps 10
Automatic Determination of Clustering Parameters 11
Clustering Sensitivity 11
Graph-Based Clustering 12
K-Means Clustering 12
Differential Expression 12
Cell Type Annotation 12
Increasing clustering sensitivity can help isolate clusters of rare cell types. On the other hand, high sensitivity leads to a higher number of clusters, which in turn can result in a less reliable gene expression profile for each cluster. How to balance this tradeoff depends on the specific purpose of each experiment. 14
Outputs 14
Metrics Extraction 14
Basic Metrics 14
Barnyard Metrics and Estimation of Multiplet Rate 15
Outputs 16
Barcoded Antibodies 17
FASTQ Processing 17
Raw Count Matrices 17
Filtered Count Matrices 18

Output Metrics 18
Tag Rank Plots 18
Feature Plots 18
Samples without Gene Expression Data 18
Cell Demultiplexing with Hashing 19
Building the negative distribution 19
Detecting multiplets with hashing 19
Diagnostic Outputs 19
Analysis of demultiplexed Matrices 20
Analysis Reports 21
Running the Full Pipeline: PIPseeker full 21
Examples 22
Input Arguments 23
General 23
FASTQ Processing 24
Mapping 25
Cell Calling 25
Barnyard Analysis 25
.Clustering 25
Cell Type Annotation 26
Report 26
ADT 26
HTO 27
Outputs 28
Rerunning Cells-only Analysis: PIPseeker cells 28
Input Arguments 29
Reprocessing a Single Demultiplexed Sample 29
Generating FASTQ files with barcodes in standard format: PIPseeker barcode 30
Generating Gene Expression Feature Plots: PIPseeker feature 30
Input Arguments 31
Generating Cell Type Annotation References: PIPseeker buildannotref 32
Input Arguments 32
Appendix 1: Colormaps 34
Document Revision Summary 34
Legal Notices 34
Support 35

Introduction

PIPseeker™ analyzes single-cell RNA data obtained with Fluent BioSciences' proprietary PIPseq™ 3' Single Cell RNA (scRNA-seq) Kits. PIPseeker offers a comprehensive analysis solution that provides

the user with detailed metrics, gene expression profiles, basic cell quality and clustering indicators, and cell type annotation for some sample types.. The outputs of PIPseeker can then be used for subsequent, specialized tertiary analysis streams. PIPseeker also supports specialized applications like measuring surface protein levels using antibody-derived tags (ADTs) and cell hashing using hashtag oligonucleotides (HTOs).

System Requirements

- PIPseeker can run on Linux, macOS and Windows operating systems.
- PIPseeker has been tested on Ubuntu 20.04 & 18.04, CentOS 7.9-2009, Windows 10 and macOS 12.3.1.
- The Windows version of PIPseeker runs from a Docker container and requires a local installation of Docker (https://docs.docker.com/get-docker). The PIPseeker executable automatically pulls the image and runs the analysis from a container.
    - Note that Docker requires a Linux backend installed on your machine. Please see this page for more information: https://docs.docker.com/desktop/install/windows-install
- Docker and Singularity images are also available for download as a standalone package and can be used to run PIPseeker on any OS.
- The required hardware settings for running PIPseeker depend on the size and characteristics of the input data and on the size of the mapping reference genome. A good rule of thumb is 2GB of RAM and 10GB of free disk space for every 1 GB of input FASTQ (in .gz format).
- PIPseeker runs a resource check before starting the analysis (It can be turned off using `--skip-preflight-check`).
- When running in a Docker or Singularity environment, make sure the virtual machine's resources are not limited in the Docker client settings.

Pipeline Overview

The PIPseeker workflow is comprised of the following steps:

1) RNA (gene expression) data processing:

a) FASTQ processing, including barcode matching, quality control and (optionally) downsampling

b) Mapping with STAR aligner

c) Parsing of the STAR output

d) Deduplication and transcript counting

e) Cell calling

f) Clustering

g) Differential expression
h) Cell type annotation (available for some sample types)
i) Metrics extraction

2) ADT data processing:
a) FASTQ processing, including barcoding, UMI error correction and ADT matching
b) Extracting ADT metrics
c) Creating merged RNA+ADT raw and filtered count matrices
d) Creating feature plots for each ADT using the RNA-based UMAP

3) HTO data processing::
a) FASTQ processing, including barcoding, UMI error correction and HTO matching
b) Extracting HTO metrics
c) Creating merged RNA+HTO (or RNA+ADT+HTO) raw and filtered count matrices
d) Demultiplexing HTO data (i.e., identify singlets, multiplets, negative cells)
e) Creating HTO plots:
i) Feature plots for each HTO using the RNA-based UMAP
ii) Normalized expression distributions (ridge plots)
iii) Scaled expression heatmap
f) Clustering for each demultiplexed sample
g) Creating ADT feature plots for each demultiplexed sample (when applicable)

4) Generating an HTML summary report

FASTQ Processing

The purpose of this step is to:

- Extract the reads that are associated with a barcode
- Remove unwanted technical sequences from the data before mapping
- Downsample the data (optional)

Structure of the FASTQ input files

PIPseq libraries are sequenced in a paired-end fashion, meaning the sequencing results will include separate FASTQ files for read 1 (R1) and read 2 (R2). Each read in R1 includes a barcode sequence, which identifies the individual PIP, followed by a molecular index sequence, identifying the specific mRNA molecule captured on each moiety. R2 includes the sequenced cDNA constructs created from the captured mRNA.

Barcode Matching

The PIPseq platform uses a tiered barcode system, where every tier includes one of a specified list of possible barcodes. Barcode matching can therefore be done highly efficiently by matching each tier in isolation. PIPseeker allows a Hamming distance of 1 per tier, i.e., the bases in the R1 FASTQ

corresponding to each tier's position can differ from a barcode by one base and still be matched to that barcode.

To simplify and reduce the size of the barcode list used by *STAR*, the tiered barcodes are replaced by a new set of generated barcodes. Intermediate FASTQ files are created with the generated barcodes and deleted at the end of the analysis.

R2 Trimming

The R2 file, which includes the cDNA construct, also includes technical sequences related to the PIPseq chemistry, namely the template switch oligo (TSO) and poly-A sequences. The prevalence of those sequences is inversely correlated with the fragment size. Because a read is less likely to be mapped if it contains extraneous sequences, we remove the TSO sequences from the 5' end and poly-A sequences from the 3' end in the R2 FASTQ file. In addition, the first base — a constant T— is removed from the 5' end (unless the TSO sequence was found there). Reads that do not meet the minimum length criterion (defined by `--min-retained-seq-length`) after these steps are discarded from further analysis.

Downsampling

In some experiments it may be desirable to standardize sequencing depth, so that experimental conditions could be reliably compared. PIPseeker allows downsampling of data to a specified number of reads using the `--downsample-to` flag. Reads are selected randomly, with the probability of selection determined by the total number of reads (provided using the `--input-reads` argument or determined automatically by counting the lines in the FASTQ inputs).

Outputs

The FASTQ processing step results in the following outputs:

- `<output path>/metrics/barcode_stats.json`: General statistics pertaining to the quality of barcode matching.
- `<output path>/metrics/barcodes/barcode_whitelist.txt`: A list of all the generated barcodes provided to *STAR* as a "whitelist" for mapping and counting.
- `<output path>/metrics/barcodes/generated_barcode_read_info_table.csv`: Detailed statistics for all the generated barcodes.
- `<output path>/barcoded_fastqs/*`: Intermediate FASTQ files to be used by *STAR* for mapping. Those files include only the subset of reads selected after the processing steps listed above. R1 files include the generated barcode for each read, followed by the molecular index, and R2 files include the original (possibly trimmed) read from the raw FASTQ input. When using multiple threads, a separate FASTQ pair will be generated by each thread. Unless instructed otherwise using the `--retain-barcoded-fastqs` flag, PIPseeker deletes those files at the end of the analysis. Note that the original, raw FASTQ files are never deleted.

Mapping

PIPseeker uses *STAR* (Spliced Transcripts Alignment to a Reference) and the associated *STARsolo* to map reads to a genome and quantify transcript abundance. *STAR* requires a special index to be built

from the genome of interest and a corresponding transcript annotation. Some common indices, such as human and mouse, can be downloaded from the Fluent BioSciences website. However, for more unique situations you may need to create your own index.

Parsing the Mapping File

STAR stores the alignment information in a Binary Alignment Map (BAM) file, which provides information about all the reads in the input FASTQ files that were mapped to the genome. The BAM file is sorted by the alignment position in the genome. PIPseeker then parses the BAM file to create a dataset with all the molecules (barcode + molecular index combinations) in the sample that were mapped to the reference. The molecule information is saved to an HDF5 format file called molecule_info.h5, and is used by PIPseeker to create the raw count (feature-barcode) matrix.

Transcriptome vs. Genome Alignment

PIPseeker distinguishes between reads mapped to a chromosome location annotated as a gene (either exon or intron, both of which are counted) and reads mapped elsewhere in the genome. If a read is not mapped to a gene, it is discarded from further processing and excluded from the count matrix. If a read is multimapped to both gene and non-gene locations, only the gene alignments are considered.

Transcript Counting

After parsing the BAM file, PIPseeker constructs a raw count matrix, commonly referred to as a feature-barcode matrix. The matrix is contained in the files: `matrix.mtx.gz`, `barcodes.tsv.gz` and `features.tsv.gz`, all inside `<output path>/raw_matrix`. These files form a sparse representation of a matrix containing a full count of unique molecules for each gene and barcode. The format of the matrix is compatible with most downstream analysis tools.

Each barcode / index combination can contribute multiple copies of the same molecule resulting from library amplification. In that case, all reads derived from the same molecule will be collapsed (deduplicated) and contribute only a single count to the raw matrix.

Multimapped Reads

PIPseeker does not allow fractional read counts, and every molecule contributes a full count to a single gene-barcode combination. However, PIPseeker uses a pseudo-multimapping approach: If a molecule is mapped to multiple genes, one of the genes will be chosen randomly, with weighted probabilities based on the frequency of mapping to each gene. Consider the following examples:

- A molecule includes a single read, mapped to gene 1 and gene 2. The count matrix will include this molecule with one of those genes, randomly selected with equal probability.
- Multimapping with deduplication: a molecule for which 5 copies (reads) mapped to both gene 1 and gene 2, and 5 other reads mapped to both gene 1 and gene 3. The total read count will be 10 for gene 1, 5 for gene 2, and 5 for gene 3. Consequently, the molecule will be assigned (and contribute a single count) to gene 1 with a 50% probability, gene 2 with a 25% probability, or gene 3 with a 25% probability.

Cell Calling

After obtaining the count matrix, barcodes are separated into putative cell-containing PIPs and background PIPs, i.e., PIPs that did not capture a cell. This separation is based on the barcode rank plot (often referred to as the "knee plot"), which orders barcodes based on the number of unique indexes, or transcripts, associated with them (Fig. 1).

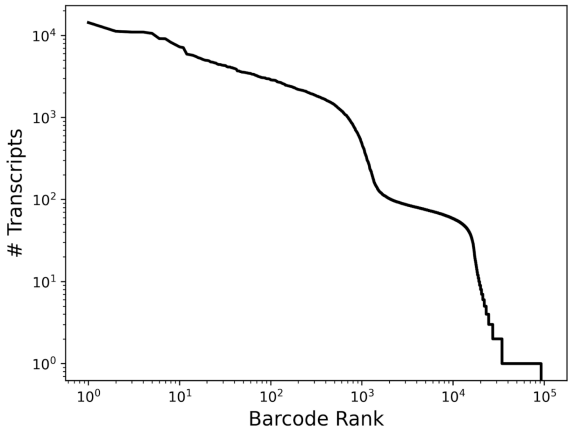


Figure 1: Barcode rank plot.

Typically, the cell barcodes are concentrated at the top mode of the rank plot, whereas the background barcodes are concentrated in the low transcript count mode. Therefore, the purpose of cell calling is to find a point in the first "knee" area that separates the two modes. This is commonly done by dividing the number of transcripts at the top of the rank plot by a constant denominator (e.g., 10). The problem with this approach is that it is highly sensitive to the shape of the rank plot, which varies strongly between different sample types and can be impacted by multiple experimental factors (e.g. viability of cell input). Fig. 2 shows the result of calling cell barcodes up to 1/10 of the top barcode's count for HEK/3T3, PBMC and breast tissue samples. The cell portion is highlighted in green.

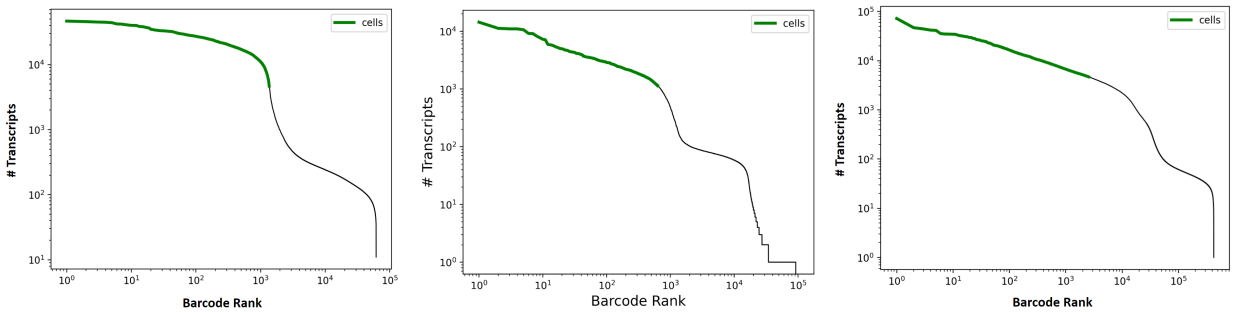


Figure 2: Cell calling up to a constant depth (1/10 of top barcode) for three different sample types of varying quality.

Cell calling for HEK/3T3 seems consistent with a visual inspection of the barcode rank plot: the entire high-transcript mode is captured. In the cases of PBMCs and breast tissue samples where the sample quality was lower, however, the number of cell barcodes is clearly underestimated. This is because the top mode has a more negative slope and is less clearly defined than in the HEK/3T3 case.

To accommodate for differences between samples and facilitate accurate cell calling, PIPseeker performs cell calling at five different sensitivity levels using the following steps:

1) Select a starting barcode, S, close to the top of the rank plot. To avoid selecting an outlier, PIPseeker detects the first barcode where the transcript count does not drop by more than 10% from the previous neighboring barcode on the plot.

2) Let M be the number of transcripts for barcode S. For each cell calling sensitivity level L_i, find a number of transcripts, T_i, such that:

$$T_i = M * 10^{-\left(0.5+0.25L_i\right)},\ i = \{1, 2, 3, 4, 5\}$$

3) For each cell calling sensitivity level L_i, select all barcodes with a transcript count higher than or equal to T_i as cell-containing barcodes.

Fig. 3 shows cell calling at five sensitivity levels for a PBMC sample.

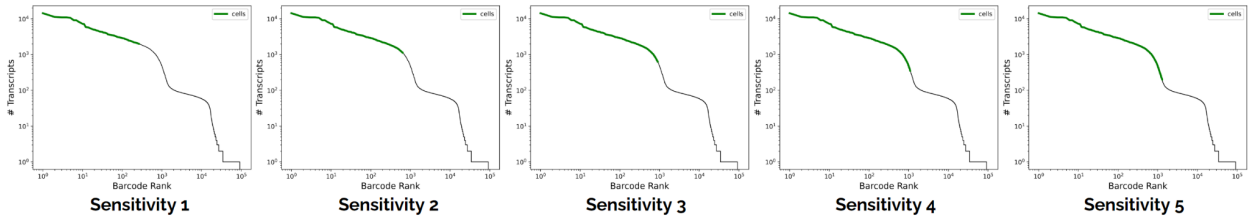


Figure 3: Cell calling with different sensitivity levels.

Cell calling inherently involves a tradeoff between the number of cells and their quality. One can arbitrarily increase the number of cells by calling deeper into the rank plot, but this increases the likelihood of selecting low-RNA content or otherwise compromised cells. The optimal number of cells recovered depends on the sample type and purpose of each experiment. For example, if some cell types in a sample (such as neutrophils or red blood cells) have low RNA expression, they may be present in high numbers in the lower transcript region of the plot. In that case, high-sensitivity cell calling may be warranted. On the other hand, if high clustering accuracy is required for precise cell type characterization, it would be preferable to use low-sensitivity cell calling. Cell calling at different sensitivity levels allows users to observe a wide range of possible outcomes and determine the correct sensitivity for their data.

Outputs

After cells are called, a directory is created for each sensitivity level inside `<output path>/cell_calling`, containing a text file with the indices of the barcodes selected as cells and an

image with a barcode rank plot. The indices start at 1 and correspond to the position of the barcodes in the file `barcodes.tsv.gz` of the raw matrix. Additionally, a filtered count matrix is generated for each sensitivity level and can be used for downstream analysis. Filtered matrices are stored in `<output path>/filtered_matrix` and contain only the cell barcodes for each sensitivity.

Manual Selection of Number of Cells

In some cases users may want more precise control of the number of called cells, beyond what is afforded with the five sensitivity levels. The `--force-cells` argument allows users to specify a desired number of cells, in which case this exact number of barcodes is selected from the top of the barcode rank plot. The outputs described above will be generated for the selected set of barcodes using the prefix force_*N*, with *N* being the number of cells input by the user..

Clustering

After selecting cell barcodes, a clustering analysis is conducted in order to quantify and visualize heterogeneity within the cell population and identify different cell types. For every set of called cells (different sensitivities or a number forced by the user), PIPseeker creates clustering maps using Leiden (graph-based) clustering, which automatically determines the number of clusters based on nearest neighbors. The user can also add K-means clustering with a specified range of values for K.

Preprocessing Steps

The starting point for clustering is the filtered count matrix, in which every row represents a cell and every column represents a gene. A number of processing steps are performed before the actual clustering algorithm is run:

1) Cell normalization: Normalize each cell's data to unit norm. This is performed so that cells with similar expression profiles but different relative RNA abundance can still be clustered together.
2) Log transform: Transform the matrix using the ln(x+1) function in order to bring it closer to a normal distribution.
3) Highly-variable gene selection: Select a subset of genes with high variance to maximize the efficiency of clustering. For every gene, the variance in expression across cells is calculated. The genes are then ranked by variance, and the top genes at the Nth percentile are selected, with N being an optional user input specified using the `--clustering-percent-genes` argument.
4) Scaling: Scale feature columns (genes) so that each column has a mean of 0 and a standard deviation of 1.
5) PCA: Use Principal Component Analysis to reduce the dimensionality of the data from the scaled set of highly variable genes to a smaller number of components that best account for variance in the expression data. The number of components can also be set automatically, as detailed below..

Following the preprocessing steps, clustering is performed in the new PCA space.

Automatic Determination of Clustering Parameters

PIPseeker determines optimal values for the number of neighbors used in k-nearest-neighbors (KNN) graph building, and for the Leiden algorithm resolution parameter. Those parameters are derived from the number of cells, N_c

$$neighbors = 2.7 * log(Nc) - 10.21$$

$$resolution = \frac{neighbors}{4*log(Nc) - 19.25}.$$

If there are not enough cells available to calculate resolution (fewer than 200), a default value of 2.5 is used.

PIPseeker also determines the number of principal components automatically using an "elbow" method, choosing the minimum value of principal components n such that

$$\frac{ev_{n+1} - ev_n}{ev_1 - ev_{200}} < ev_{200}$$

where ev_n is the explained variance of the nth principal component.

If less than 200 components are available, the number of components used is the minimum of the number of cells – 1, number of genes – 1, and 15 components.

To override automatic selection of clustering parameters, PIPseeker requires the user to specify `--nearest-neighbors` for the KNN graph, `--resolution` for the Leiden algorithm, and `--principal-components` for the number of components used in PCA.

Clustering Sensitivity

PIPseeker supports clustering with sensitivities 'low', 'medium', and 'high', which can be set using `--clustering-sensitivity` (note that clustering sensitivity cannot be set when overriding automatic selection of clustering parameters). The number of clusters detected by Leiden will increase with clustering sensitivity. By default, PIPseeker will cluster using 'medium' sensitivity, where clustering parameter values are calculated using the previously described optimized equations. In 'low' sensitivity mode, the value of the resolution parameter in Leiden is decreased by 40% and the value of the nearest neighbors parameter in k-nearest-neighbors graph building is increased by 40% compared to the default 'medium' sensitivity values. 'high' clustering sensitivity increases the value of the resolution parameter by 40% and decreases the value of nearest neighbors by 40% compared to the 'medium' sensitivity values.

In 'low' sensitivity mode, fewer communities are detected, resulting in fewer overall clusters which contain more cells. Conversely, 'high' sensitivity results in more communities overall and allows for smaller possible community size.

Graph-Based Clustering

PIPseeker performs clustering using the Leiden algorithm, the current standard for graph-based clustering in scRNA-seq datasets. A k-nearest-neighbor (KNN) graph is constructed from the PCA matrix, which is converted to a directed node-edge graph and passed into Leiden. The Leiden algorithm uses a modularity optimization of graph communities to determine ideal cluster membership. Leiden has the added advantage of minimal user inputs (e.g., number of clusters, cutoffs, etc.) compared to other methods. As noted above, the user can control some of the Leiden parameters (`--nearest-neighbors`, `--resolution`) or have them be determined automatically.

K-Means Clustering

PIPseeker also uses the popular K-means algorithm to divide cells into distributed clusters. It runs the algorithm for different values of K, representing a predetermined number of clusters. The minimum and maximum of the range of K values can be controlled by the user (`--min-clusters-kmeans` and `--max-clusters-kmeans`) and should be tailored to the number of cell types expected to be present in the sample.

Differential Expression

Once clustering is completed, PIPseeker finds the differentially expressed genes associated with each cluster. For this step, PIPseeker uses the Wilcoxon rank-sum test for independent variables (Mann-Whitney U). This nonparametric test is used to determine whether two samples come from the same distribution by ranking the values associated with two groups together, and calculating the sum of those ranks within each group. Those ranks are used to calculate a test statistic (U), which PIPseeker uses to create a rank order of differentially expressed genes for each cluster.

For each cluster, PIPseeker computes differential expression for genes associated with cells across that cluster vs. all other cells. The top genes, sorted by descending Z-score, are exported to a CSV file. The number of reported genes is controlled by the `--diff-exp-genes` argument. This process is performed for different values of K in K-means clustering, as well as for graph-based Leiden clustering.

Cell Type Annotation

For certain sample types, PIPseeker can assign cell types to clusters based on the similarity in gene expression to a known reference dataset. The user input `--annotation` points to a reference file, which can be downloaded from the Fluent BioSciences website or generated by PIPseeker buildannotref.

Cell types are assigned based on the correlation between differential expression scores for each cluster and the scores for the annotated types in the reference dataset If the correlation with a certain type is high (r >= 0.5), the cluster is assigned that type as “Primary”. If there is high correlation with another type (r >= 0.3), the second best-matching cell type gets assigned as a “Secondary”. The primary cell types are then used to generate a UMAP plot colored by cell types. Clusters with low similarity to all reference cell types are not assigned and are labeled “Unknown”..

Clustering sensitivity levels can be used in combination with cell type annotation to optimize clustering outputs. In the example in Fig. 4, 23,779 PBMC cells were clustered using 'low', 'medium', and 'high' clustering sensitivity modes. The number of clusters and detected cell types increase as clustering sensitivity increases. In 'low' sensitivity mode, 10 cell types are identified from 13 detected clusters. This mode best maintains the Naïve CD4 T cell cluster, spreading this cell type across three clusters, compared to four clusters in 'medium' sensitivity mode and five clusters in 'high' sensitivity. In 'medium' sensitivity mode, an additional Central Memory CD4 T cell cluster is detected, Finally, 'high' clustering sensitivity reveals a cluster of rare stem cells (HSPC).

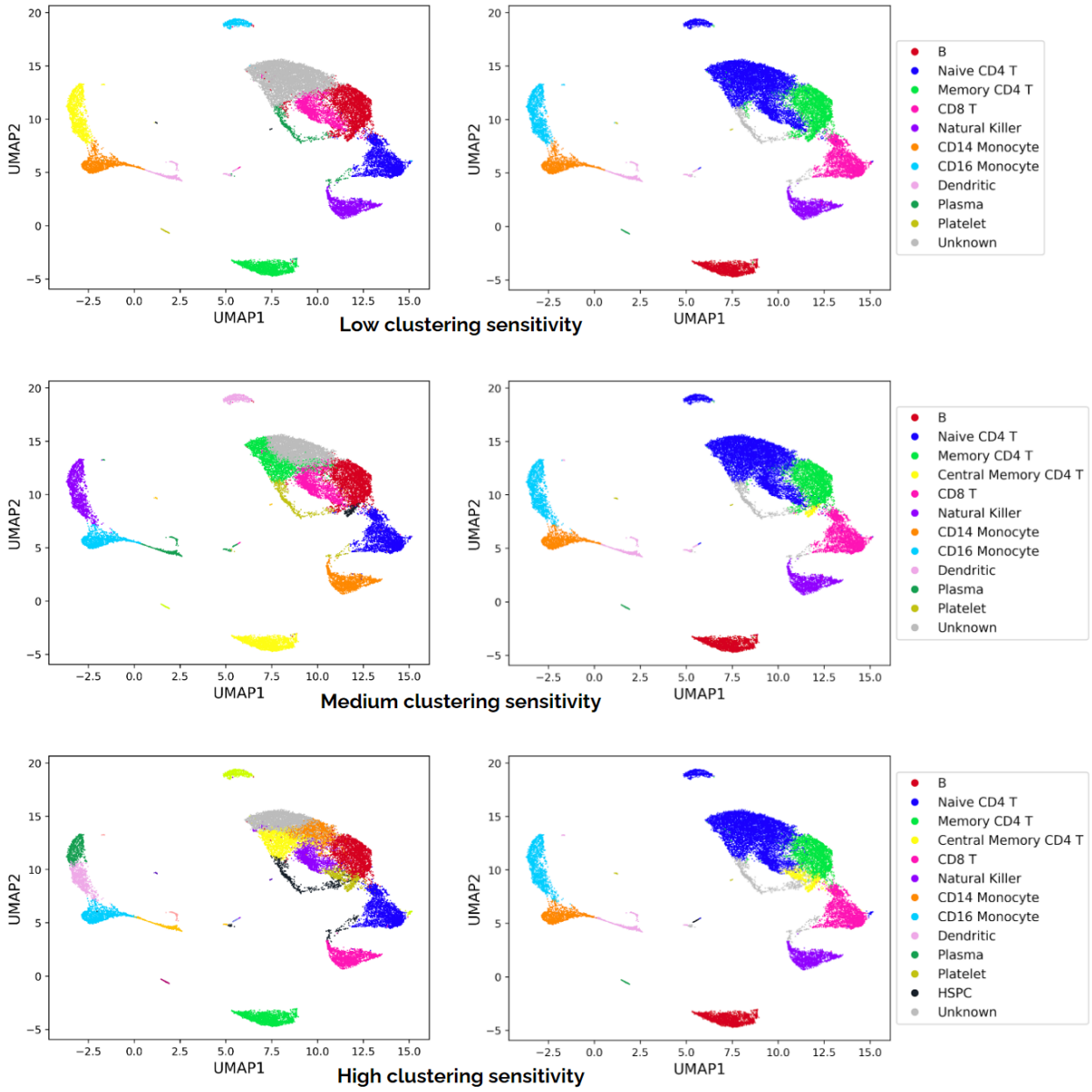


Figure 4: Graph-based clustering and cell type annotation for a PBMC sample at three different clustering sensitivities. The left panels show graph-based clustering results, and the right panels show the corresponding cell type annotations, as indicated in the legends. Note the increase in the number of clusters and in the detection of additional cell types as sensitivity increases. Also note that multiple clusters can get assigned the same cell type, and some clusters do not get assigned a cell type ("Unknown").

Increasing clustering sensitivity can help isolate clusters of rare cell types. On the other hand, high sensitivity leads to a higher number of clusters, which in turn can result in a less reliable gene expression profile for each cluster. How to balance this tradeoff depends on the specific purpose of each experiment.

Outputs

A directory is created for each sensitivity level inside `<output path>/clustering`, containing the following:

- `clusters.csv`: cluster assignment of each cell from graph-based clustering and from all the K-means runs.
- `umap/*.png`: UMAP plots of clusters. UMAP (Uniform Manifold Approximation and Projection) is a popular method for projecting multi-dimensional data onto a 2-D space.
- `top_genes/*.csv`: top-ranked differentially-expressed genes for each cluster.
- `differential_expression/*.csv`: top-ranked differentially expressed genes for each cluster, with additional statistics for each gene: Z-score, p-value, adjusted p-value and log2 fold change.
- When cell type annotation is used:
    - `cell_type_annotation/<sample type>/cell_types/*.csv`: cell type assignments for each cell (barcode ID) and for each cluster.
    - `cell_type_annotation/<sample type>/umap/*.png`: cell type maps for each clustering type.

Metrics Extraction

In the next step of the analysis, various metrics are calculated and reported for each sensitivity level (or for a single set when the number of cells is specified by the user).

Basic Metrics

The following metrics are produced in every analysis and output for each sensitivity level::

- Total number of reads in the input FASTQ files.
- Total number of reads above minimum length after R2 trimming.
- Total number of reads for which valid barcodes were detected (reads sent to *STAR* for mapping).
- Total number of reads mapped to the transcriptome.
- Percentage of mapped reads
    - Percentage of transcriptome-mapped reads out of the total number reads provided to STAR after FASTQ processing (reads with valid barcodes).
- Percentage of reads in cells

- Percentage of reads associated with cell-containing barcodes out of the total number of input reads.
- Number of called cells
- Mean reads per cell
  - The total number of input reads divided by the number of cells.
- Duplication rate
  - The number of total mapped reads in cells divided by the number of unique (deduplicated) reads in cells.
  - For example: If, on average, every molecular index is associated with three reads, i.e., two deduplicated reads per index, the duplication rate would be 3.
- % sequencing saturation
  - The percentage of the total cDNA library that has been observed at a given sequencing depth. Also indicates the probability that when sequencing an additional read, the library molecule would have already been detected.
  - Defined as: $100 \star (1 - \frac{1}{dup.rate})$.
- Total number of transcripts in cells
- Median transcripts per cell
  - This is based on the distribution of counts of unique indexes (transcripts) associated with each cell barcode.
- Number of genes expressed in cells
  - This number includes all genes with a nonzero count in the filtered matrix, and gives equal weight to every gene regardless of expression level.
- Median genes per cell
- Percentage of mitochondrial transcripts in cells
  - This is based on gene names and may not work with every mapping reference. Any analysis with a reference provided by Fluent BioSciences will include this metric.

Barnyard Metrics and Estimation of Multiplet Rate

One common way to assess the quality of scRNA-seq data is a combined species ("barnyard") experiment, typically involving a mixture of human and mouse cells. Such an experiment can provide information about the prevalence of multiplets, i.e., PIPs that capture more than a single cell. It can also assess the degree of interference from ambient RNA by measuring cross-species contamination. The more human genes found in mouse cells and vice versa, the more ambient RNA contamination is likely present in the system.

When the reference transcriptome is a combined human and mouse transcriptome, and the `--run-barnyard` flag is used, PIPseeker can provide an additional set of metrics specific to a barnyard experiment:

- Number of human cells
  - The number of cell barcodes with at least 85% of the transcripts mapping to the human reference.
- Number of mouse cells
  - The number of cell barcodes with at least 85% of the transcripts mapping to the mouse reference.

- Number of multiplets
  - The number of cell barcodes with less than 85% of the transcripts mapping to the human or the mouse reference.
  - Note that "hidden" multiplets can also occur when two or more cells of the same species are captured in a single droplet. Currently, PIPseeker only reports the number of observed multiplets and does not include a prediction for hidden multiplets that may be present in the sample.
- Total number of transcripts in human cells
- Median human transcripts per human cell
- Number of human genes expressed in human cells
- Median human genes per human cell
- Percentage of mouse transcripts in human cells
- Total number of transcripts in mouse cells
- Median mouse transcripts per mouse cell
- Number of mouse genes expressed in mouse cells
- Median mouse genes per mouse cell
- Percentage of human transcripts in mouse cells

An important output of a barnyard analysis is called a barnyard plot (Fig. 5). It shows the number of mouse transcripts vs. the number of human transcripts, with different colors indicating human cells, mouse cells and multiplets:

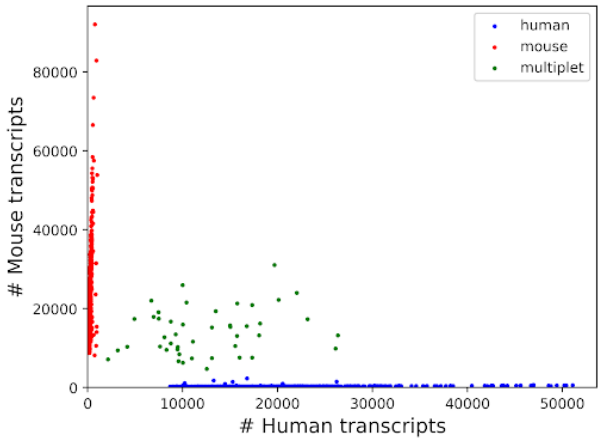


Figure 5: "Barnyard" plot.

Outputs

After all the metrics are calculated, results for each sensitivity level are reported in the following files:

- Basic metrics are stored in csv and json formats in `<output path>/metrics`.
- If applicable, barnyard metrics are stored in csv and json formats in `<output path>/barnyard`.

Barcoded Antibodies

In addition to gene expression based on sequences derived from captured mRNA, PIPseeker can also process sequencing data from tags attached to cell surface-bound antibodies. This technique is used in applications like CITE-seq, which uses sequencing to measure surface protein expression, and cell hashing, which is used to multiplex cell samples and detect multiplets.

Barcoded antibodies are sequenced separately from the cDNA sample. While cells are identified using tiered cell barcodes in exactly the same way, the R2 file is not mapped to a reference genome. Instead, sequences are matched against a file containing the tag sequences attached to the antibodies. Reads are then deduplicated and counted to produce count matrices similar to those derived from gene expression data.

Tags used to measure surface expression are referred to as Antibody-Derived Tags (ADTs), and those used for multiplexing are named Hashtag Oligonucleotides (HTOs). PIPseeker can process either type or both in the same analysis. The initial processing is similar, but some outputs are specific to each technique.

FASTQ Processing

After completing the gene expression analysis, PIPseeker can process additional FASTQ files provided using `--adt-fastq` and/or `--hto-fastq`. It matches reads in R2 to sequences in files specified by `--adt-tags` and `--hto-tags`. Tag files are CSV format files (no header), with columns for sequence, tag ID and (optionally) a tag description, as in the example below:

```
ACTGTA,Tag 1,Tag 1 description
CCTGTA,Tag 2,Tag 2 description
CACCGT,Tag 3,Tag 3 description
ACGCTA,Tag 4,Tag 4 description
TCAACT,Tag 5,Tag 5 description
:
```

Cell barcodes are matched in a tiered fashion just like the RNA data. The molecular indexes are then matched with an error tolerance of 1. Each index is collapsed into the most commonly occurring index from which it is one base apart, if such indexes exist. Otherwise, the index is retained as a distinct molecular identifier. Finally, the R2 reads are matched against the sequences in the tags file, again with an allowed Hamming distance of 1.

Raw Count Matrices

The FASTQ processing step results in a raw count matrix for the tags. It is similar to a gene expression count matrix, only here the features are antibody tags. This matrix is then merged with the RNA raw matrix to produce a single matrix containing both genes and tags as features. The barcodes in the tags matrix renumbered to match those of the RNA matrix, and those that are unique to the tags matrix are appended after the RNA matrix's barcode list. The tags in the feature file are appended after the genes to produce a merged features file.

The following raw matrix outputs are generated in the output directory:

- ADT: `ADT/adt_raw_matrix`, `ADT/raw_matrix` (merged)
- HTO: `HTO/hto_raw_matrix`, `HTO/raw_matrix` (merged)
    - If both ADT and HTO are included, the merged HTO matrix will include ADT features.

Filtered Count Matrices

Like in gene expression analysis, the merged raw matrices are subset to produce filtered matrices that only contain cell barcodes. The cell barcodes are the same ones selected in the cell calling step based on RNA data. The filtered matrices are stored in `ADT/filtered_matrix` and/or `HTO/filtered_matrix`.

Output Metrics

:

- Total number of reads in the input FASTQ files..
- Total number of reads for which valid barcodes were detected.
- The number of reads with valid barcodes that were successfully matched to one of the tags.
- Duplication rate (see description for RNA-based analysis).
- % sequencing saturation (see description for RNA-based analysis).
- Total deduplicated read count in cells
- Median deduplicated read count in cells
- Median number of unique tags in cells

Metrics are saved for each cell calling sensitivity in `ADT/metrics` or `HTO/metrics`. In addition, a text file with the total (deduplicated) count for each tag is saved in a directory called `counts_per_tag`.

Tag Rank Plots

LIke with RNA, PIPseeker produces barcode rank plots for each tag modality, ADT and/or HTO, using the raw tag matrix. Rank plots are annotated by the cell barcodes called by RNA-based cell calling. The plots are saved as `ADT/plots/barcode_rank/<sensitivity level>.png` or `HTO/plots/barcode_rank/<sensitivity level>.png`.

Feature Plots

The tag counts are normalized using center-log ratio (CLR) transformation. A color gradient representing tag expression is overlaid in UMAP space with locations based on the results of cell clustering. Note that the clustering is based on gene expression, not tag counts. The plots are saved into `ADT/plots/feature` or `HTO/plots/feature`.

Samples without Gene Expression Data

PIPseeker can process ADT data without an accompanying RNA count matrix. In this case, the analysis will stop after generating the ADT raw matrix. If cell hashing is included, a merged ADT + HTO raw matrix will be created. Cell calling and all subsequent steps will not be performed.

Cell Demultiplexing with Hashing

Cell hashing is typically used to combine distinct samples into a single run and later separate them based on different barcoded antibody sequences attached to each sample. Separating, or demultiplexing, the samples involves building a negative distribution for each HTO in order to define a threshold of expression beyond which a cell can be confidently assigned to a sample. Each HTO is assigned in this manner to a single sample or determined to match more than one sample (multiplet) or none at all.

Building the negative distribution

To define a negative distribution for each hash, K-means clustering is performed on the CLR-normalized HTO matrix. The number of clusters is set as one more than the number of hashes in order to produce an "unassigned" cluster in addition to a cluster for each hash. For each hash, the array of normalized expression values is separated into distributions associated with each cluster. This includes the unassigned cluster, which presumably contains all cells that do not have a putative label. The cluster with the lowest average expression for that hash is termed the *negative* distribution. The 99th percentile of that negative distribution becomes the threshold for determining whether a cell is positive or negative for that hash. Normally, the negative distribution will be the unassigned cluster, but sometimes it can be another hash distribution if the unassigned cluster has elevated expression.

PIPseeker outputs "ridge" plots (Fig. 6A) for each hash with the normalized expression value distributions of each cluster (row in the plot) and the threshold defined from the negative distribution. Plots for each hash are saved as `<hash ID>.png` into directories for each sensitivity level in `HTO/plots/ridge`.

After performing the same analysis for each hash, the total number of *positives* are calculated for each cell. If a cell is positive for no hashes, it is determined to be in a global negative category. If a cell is positive for more than one hash, it is determined to be in a global multiplet category. Most cells should be positive for only a single hash and placed in a global singlet category.

Detecting multiplets with hashing

As noted above, some cells receive a "multiplet" designation. It is important to note, however, that hashing cannot determine whether these cells have more than one cell's worth of material. It simply means that a cell has higher expression than the negative distribution threshold in more than one hash, which is also consistent with a global *ambiguous* category. When cells are not labeled well with hashes, the "multiplet" category is often highly represented by ambiguous calls because of the low signal-to-noise ratio. In a successful hashing experiment, most of the cells should be positive for only one hash, putting them in a global singlet category. In this case, the singlet global category can be further separated into each hash.

Diagnostic Outputs

To visualize the cell distribution, PIPseeker generates a heatmap (Fig. 6B) of CLR-normalized expression with column values sorted into singlet, multiplet, and negative global categories (left to

right), and separated by black lines. The singlet order is further separated into the order of the assigned hashes (top to bottom), and sorted by expression of the assigned hash. Multiplets and negatives are sorted by total expression across all hashes. A color legend is appended to the top of the heatmap, with colors corresponding to the hash names on the y-axis. The white bar represents multiplets, while the gray represents negatives. The heatmap is saved as `<sensitivity level>.png` into directories for each sensitivity level in `HTO/plots/heatmap`.

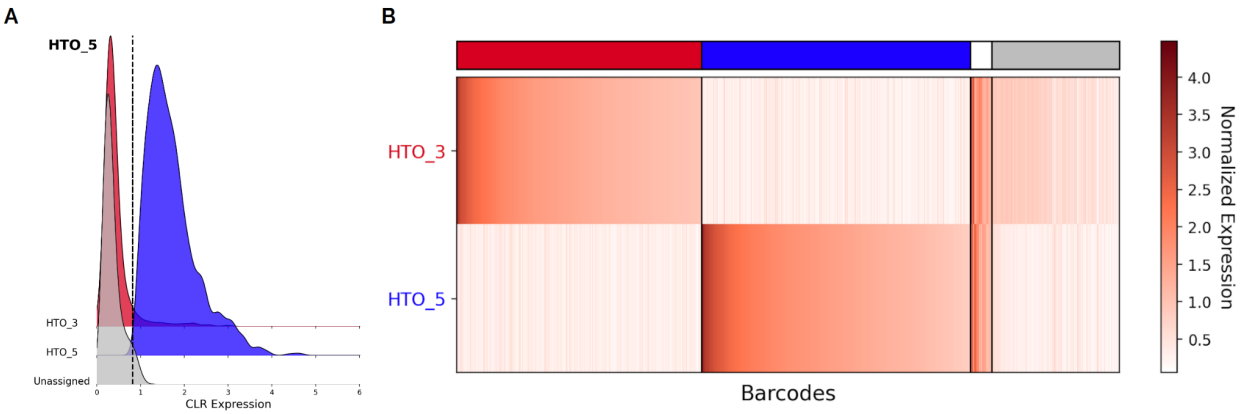


Figure 6: HTO output examples. A: Ridge plot for a single hash. B: Hashing heatmap..

PIPseeker outputs a `demultiplexing.csv` file into sensitivity level directories in `HTO/demux` with the following information associated with each cell:

1. Hash global classification: singlet, multiplet, or negative
2. Hash ID: hash ID, multiplet, or negative
3. Max ID: hash associated with maximum CLR-normalized expression, regardless of PIPseeker hash global classification
4. Hash max: the maximum CLR-normalized expression value
5. Hash second max: the second highest CLR-normalized expression value
6. Margin: the difference between the max and second max

Analysis of demultiplexed Matrices

After demultiplexing, PIPseeker divides the RNA matrix into separate matrices for each singlet hash identity in `HTO/demux/<sensitivity level>`. Those matrices are then used for further analysis of each demultiplexed sample. PIPseeker creates clustering outputs for each demultiplexed sample, using the same clustering parameters that were used for the combined sample. If ADT data is included, feature plots will be generated for each demultiplexed sample. The outputs for each demultiplexed sample are included in a separate section of the analysis report.

The user may want to reprocess a demultiplexed sample with different parameters than were used for the combined sample. Some examples are increasing the clustering sensitivity, adding K-means clustering, or changing the ADT colormap in order to have a specific color for each sample. Use the `--hash` input with PIPseeker cells to reanalyze a single demultiplexed sample.

Analysis Reports

Every run of PIPseeker produces a report in HTML format, `<output path>/report.html`, which include basic metrics, a barcode rank plot, a clustering map, a differential expression table, and, if applicable, a cell type annotation map. If "barnyard" mode is used, those statistics and a barnyard plot will be included. If ADT or HTO data were included in the analysis, those results will also be incorporated into the report.

The report contains a tab for each cell calling sensitivity level, as well as any analyses performed with a forced number of cells. The user can browse through the different sensitivities and select the optimal sensitivity level or decide on a number of cells to force (`--force-cells`) in a subsequent reanalysis (PIPseeker cells).

In the case of an analysis with a manually selected (forced) number of cells, a report will be generated for this specific number, and a new tab will be added to the combined report.

Running the Full Pipeline: PIPseeker full

PIPseeker runs from the command line prompt on Linux, Mac and Windows. A Quick Start tutorial is available here: https://www.fluentbio.com/resources/pipseeker-tutorial

- On Mac and Linux, make sure the PIPseeker file has executable permission. You can change its permissions using: `chmod +x <PIPseeker file>`
    - Mac users may need to provide the system additional permissions before the executable can be run.
- If you downloaded a mapping reference from the Fluent BioSciences website, make sure it is uncompressed (both the .tar and the .gz components). It should be a directory.
- Optionally, you can add the PIPseeker executable to the system path. After you do that, you will not need to specify the full path to it, but just to type `pipseeker`.
    - Download and unpack the PIPseeker package and move it to a permanent location.
    - Linux/Mac:
        i. Add the pipseeker executable `$PATH` by running the following command: `export PATH=/path/to/pipseeker` (the actual path depends on where you moved the executable).
        ii. To permanently modify the system path, add the above command to your login script, e.g., `.bashrc` or `.cshrc`. On Mac, the will be `.zshrc` if using the default terminal settings.
    - Windows:
        i. Right-click on the Start Button and select "System".
        ii. Click "Advanced system settings".
        iii. Go to the "Advanced" tab and select "Environment Variables".
        iv. Double-click the variable called "Path" and choose "Edit".

v. Click “New”.
vi. Enter the path to the folder containing PIPseeker and select “Ok” on all the settings windows you opened.
- You may need to restart your terminal or PowerShell for changes to the path to take effect.

- On Windows, run PIPseeker from a Command Prompt or from PowerShell. PIPseeker is not a clickable GUI application.
- File and directory names containing spaces must be enclosed in quotation marks.
- If running from a Docker or Singularity container, make sure to mount any of the following directories (depending on the type of analysis) onto the container space:
  - Input directory
  - Output directory
  - STAR index directory
  - Directory containing the cell type annotation file
  - Directories containing ADT or HTO FASTQ files
  - Directories containing ADT or HTO tag files
- If running on Windows and any of your input, index or output directories are on a network drive, you may need to explicitly allow that drive to be mounted using the Docker client settings.

Examples

Show the full list of arguments:

```
$ <path to PIPseeker>/pipseeker full --help.
```

Standard gene expression analysis without changing any of the default arguments:

```
$ <path to PIPseeker>/pipseeker full --fastq <FASTQ files prefix>
--star-index-path <path to mapping reference>
```

Gene expression analysis with high-sensitivity graph-based clustering and K-means clustering with 4 or 5 clusters:

```
$ <path to PIPseeker>/pipseeker full --fastq <FASTQ files prefix>
--star-index-path <path to mapping reference> --clustering-sensitivity
high --min-clusters-kmeans 4 --max-clusters-kmeans 5
```

Gene expression + ADT analysis, with ADTs starting at the 5th R2 cycle :

```
$ <path to PIPseeker>/pipseeker full --fastq <FASTQ files prefix>
--star-index-path <path to mapping reference> --adt-fastq <ADT FASTQ file
prefix> --adt-tags <path to ADT tags CSV file> --adt-position 4
```

Input Arguments

General

`--fastq` (string)

A string containing the path to the source directory for input FASTQ files and a prefix for the names of the files to be processed. Replace the prefix with a period (`.`) to indicate that all files in the directory should be processed..

For example, consider a situation where the input directory contains two-lane results from two different samples:

```
$ ls my_fastq_dir
sample_1_L001_R1.fastq.gz      sample_1_L001_R2.fastq.gz
sample_1_L002_R1.fastq.gz      sample_1_L002_R2.fastq.gz
sample_2_L001_R1.fastq.gz      sample_2_L001_R2.fastq.gz
sample_2_L002_R1.fastq.gz      sample_2_L002_R2.fastq.gz
```

Use `--fastq my_fastq_dir/sample_1` and `--fastq my_fastq_dir/sample_2` in two separate runs to include only the relevant files for each sample.

`--star-index-path` (string)

The path to a directory containing an indexed reference to use for mapping.`--output-path` (string)

The directory where all PIPseeker outputs will be stored. If not specified, will be set to the same directory as the directory containing the input FASTQ files.

`--verbosity` (integer, default: 1)

Verbosity level: 0 (silent), 1 (concise), or 2 (detailed).

`--chemistry` (string)

Version of the PIPseq assay (v3 or v4). If not specified, PIPseeker will attempt to automatically detect the chemistry version based on the read structure in the input files.

`--skip-version-check`

Do not check for newer versions of PIPseeker on the Fluent BioSciences website..

`--skip-preflight-check`

Skip the system checks for sufficient resources prior to data processing.

`--resume-last-run`

Resume processing from the last completed step in the output directory, instead of restarting from the raw FASTQs. This is useful for recovering from a run that failed for any reason, such as running out of disk space or losing power.

`--threads` (integer, default: 8)

Number of threads to use for parallel processing. Allowed values:

- Exact number of threads
- 1 for no parallel processing
- 0 to auto-detect and utilize all available cores

Note that the number of threads for *STAR* is controlled separately.

`--random-seed` (non-negative integer, default: 0)

Seed for random number generation. This affects raw matrix generation (when resolving multimapping) and clustering.

`--dpi` (integer, default: 200)

DPI (dots per inch) resolution for all saved figures..

`--save-svg`

Save copies of all generated plots in Scaled Vector Graphics (SVG) format. By default, PIPseeker only saves plots only in Portable Network Graphics (PNG) format.

FASTQ Processing

`--downsample-to` (integer)

An integer representing the target number of reads to be included if downsampling is desired. Reads will be selected randomly and only the selected subset will be considered for barcode matching.

`--input-reads` (integer)

An integer representing the number of input reads if `--downsample-to` is specified. In order for PIPseeker to calculate the downsampling ratio, the total number of reads must be known in advance. `--input-reads` should include the total number of reads combined across all input files. If it is not specified, PIPseeker will count the lines in the input files, so using this argument can reduce running time.

`--retain-barcoded-fastqs`

A flag for retaining the intermediate, barcoded FASTQ files. If not specified, these files will be discarded after completion of the analysis. Note that the original input FASTQ files are always retained, regardless of this flag.

Mapping

`--star-path`

Path to a *STAR* binary executable to use for mapping. Usually this argument is not necessary, since a distribution of *STAR* is packaged within PIPseeker.

`--star-threads` (integer, default: 8)

Number of CPU threads to use for *STAR* mapping.

`--remove-bam`

Remove the BAM file STAR output after run completion.

Cell Calling

`--force-cells` (integer)

Force a specific number of barcodes to be considered as cells. If specified, this number of barcodes will be selected from the top of the barcode rank plot. This will override the default mode of cell calling using sensitivity levels.

Barnyard Analysis

`--run-barnyard`

Enable “barnyard” metrics to be generated and included in the analysis report

.Clustering

`--clustering-percent-genes` (float, default: 10)

Percentage of genes to use for clustering. Genes with the highest variability in expression up to this percentile rank will be included.

`--diff-exp-genes` (integer, default: 50)

Number of top differentially expressed genes to report for each cluster.

`--principal-components` (integer)

The number of components for PCA dimensionality reduction.

`--nearest-neighbors` (integer)

Number of nearest neighbors for graph-based clustering. This defines the minimum number of cells per cluster.

`--resolution` (float)

Resolution parameter for graph-based clustering..

Note: --principal-components, --nearest-neighbors and --resolution must all be used or omitted at the same time. You cannot specify one argument and leave the others unspecified. If none of the arguments are specified, PIPseeker will assign them automatically.

`--clustering-sensitivity` (low, medium or high, default: medium)

Sensitivity level for graph based clustering.

`--min-clusters-kmeans`, `--max-clusters-kmeans` (integer)

Minimum and maximum number of clusters for K-means clustering. By default, no K-means clustering is performed.

Cell Type Annotation

`--annotation` (string)

Optional path to a file to use as reference for cell type annotation of the clustering results. Reference files are available to download from the Fluent BioSciences website..

Report

`--id` (string)

Sample ID to include in the output report.

`--description` (string)

Sample description to include in the output report.

ADT

`--adt-fastq` (string)

A string containing the path to the source directory for ADT FASTQ files and a prefix for the names of the files to be processed. Replace the prefix with a period (`.`) to indicate that all files in the directory should be processed..

`--adt-tags` (string)

A path to a CSV file containing ADT sequences and labels.

`--adt-position` (integer, default: 0)

Starting position of the ADT sequence in the R2 FASTQ file (cycle 1 is position 0).

`--adt-colormap` (string, default: gray-to-green)

Colormap for ADT feature plots. See appendix 1 for available colormaps.

HTO

`--hto-fastq` (string)

A string containing the path to the source directory for HTO FASTQ files and a prefix for the names of the files to be processed. Replace the prefix with a period (`.`) to indicate that all files in the directory should be processed..

`--hto-tags` (string)

A path to a CSV file containing HTO sequences and labels.

`--hto-position` (integer, default: 0)

Starting position of the HTO sequence in the R2 FASTQ file (cycle 1 is position 0).

`--hto-colormap` (string, default: gray-to-red)

Colormap for HTO plots. See appendix 1 for available colormaps.

Outputs

Fig. 7 is an overview of the structure and content of the output directory from PIPseeker full. This output can be used for subsequent workflows, as described below.

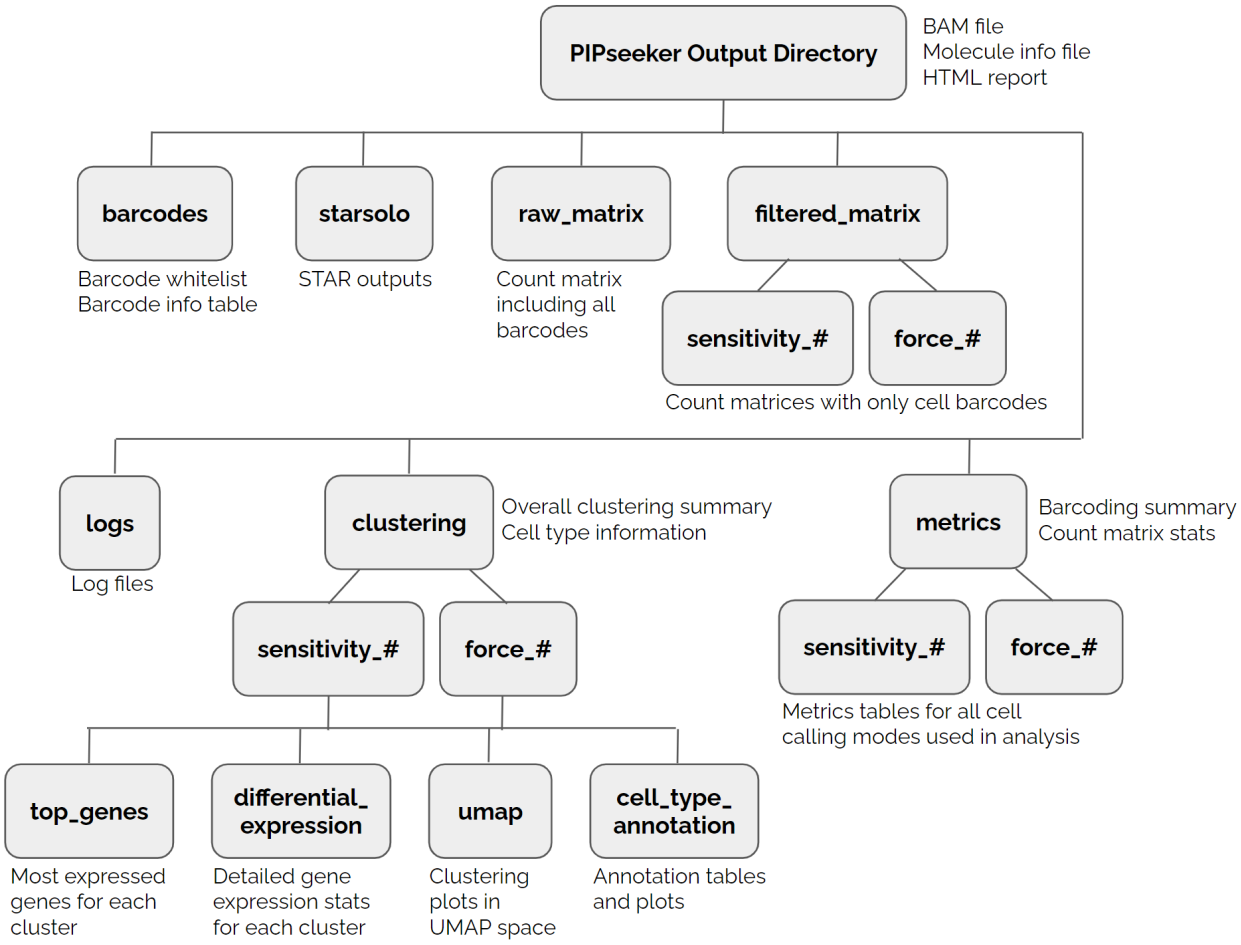


Figure 7: PIPseeker full outputs. Note that this diagram does not include specialized workflows, such as Barnyard, ADT and HTO.

Rerunning Cells-only Analysis: PIPseeker cells

After observing the results of a complete analysis, you may wish to change a parameter related to the late stages involving cells, You may also wish to force a certain number of barcodes to be designated as cells, or change other parameters such as the range of K values for K-means clustering or the number of differentially expressed genes to display. This does not require repeating all the processing involved in creating the raw matrix, which can be time consuming.

`pipseeker cells` takes as input the path to a previously generated PIPseeker result, as specified by the `--previous` argument.. This will typically be the output directory from a prior `pipseeker full` run. For example, assume that after running the full pipeline and visually inspecting the results, none of the standard cell-calling sensitivity levels seem to capture the correct region of the barcode rank plot. You can force a specific number of cells without re-running the entire pipeline:

```
$ <path to PIPseeker>/pipseeker cells --previous my/analysis/result/dir
--force-cells 5000
```

The results from the new analysis will be added to the directory previously used with `PIPseeker full`. A new tab representing the PIPseeker cells run (`force_5000` in the above example) will be added to the HTML report containing the results for the original five sensitivities..

Input Arguments

All arguments described above for `PIPseeker full` are applicable in PIPseeker cells mode, except for those related specifically to the steps prior to building the raw count matrix..

`--previous` (string)

A path to a directory containing the result of a previously completed PIPseeker analysis. The raw matrix and other information will be loaded from this location.

`--hash` (string)

A hash label of an individual demultiplexed sample from cell hashing.

Reprocessing a Single Demultiplexed Sample

When using cell hashing to combine and demultiplex samples, it may be desirable to analyze different samples with different parameters. As part of the initial PIPseeker full run with HTOs, PIPseeker already processed each demultiplexed sample with the same parameters as were used to process the combined samples. Using PIPseeker cells with the `--hash` argument will start a reanalysis of only a single demultiplexed sample:

```
$ <path to PIPseeker>/pipseeker cells --previous my/analysis/result/dir
--clustering-sensitivity high --min-clusters-kmeans 3 --max-clusters-kmeans 5
--hash <HTO label>
```

The command above will reprocess a specific demultiplexed sample with higher clustering sensitivity than the default, and will add K-means clustering in addition to graph-based clustering. Results will be output into the `HTO/demux` directory under the main results directory. Note that `--previous` should refer to the top results directory from a previous analysis, not a demultiplexed sample directory.

Generating FASTQ files with barcodes in standard format: PIPseeker barcode

As noted above, the PIPseq platform uses tiered barcodes, which are converted into contiguous 16-base barcodes in intermediate FASTQ files, that are then used for mapping. Some custom workflows may require those intermediate barcodes, for example when running your own version of STAR or a custom aligner.

`pipseeker barcode` generates FASTQ files and a barcode whitelist compatible with the standard format expected by most scRNA-seq processing tools. It stops after completing the barcoding step. The input parameters and the processing up to that point are identical to `pipseeker full`.

Generating Gene Expression Feature Plots: PIPseeker feature

A common workflow following gene expression analysis is to plot expression levels for individual genes over the UMAP output of the clustering step. This can highlight differences in gene expression between cell populations. PIPseeker can generate any number of these plots based on previous analysis results.

`pipseeker feature` takes as input the path to a previously generated PIPseeker result, as specified by the `--previous` argument.. It also requires a list of one or more genes, either provided directly in the command line or included in a file:

```
$ <path to PIPseeker>/pipseeker feature --previous my/analysis/result/dir
--genes Gene_1,Gene_2
```
or:
```
$ <path to PIPseeker>/pipseeker feature --previous my/analysis/result/dir
--genes-file genes.txt
```

Feature plots will be output to a directory named `feature_plots` in the directory defined by `--previous`, with one set of plots for each sensitivity level present in the original run.

Fig. 8 shows examples of feature plots from two samples. Panel A shows cell type annotation and two genes in a human PBMC sample: MS4A1, which is typically expressed in B cells, and LYZ, a monocyte marker. Panel B shows a mouse brain sample, with plots for genes Prdx6 and Scn4b, markers for astrocytes and striatal D1 medium spiny neurons, respectively.

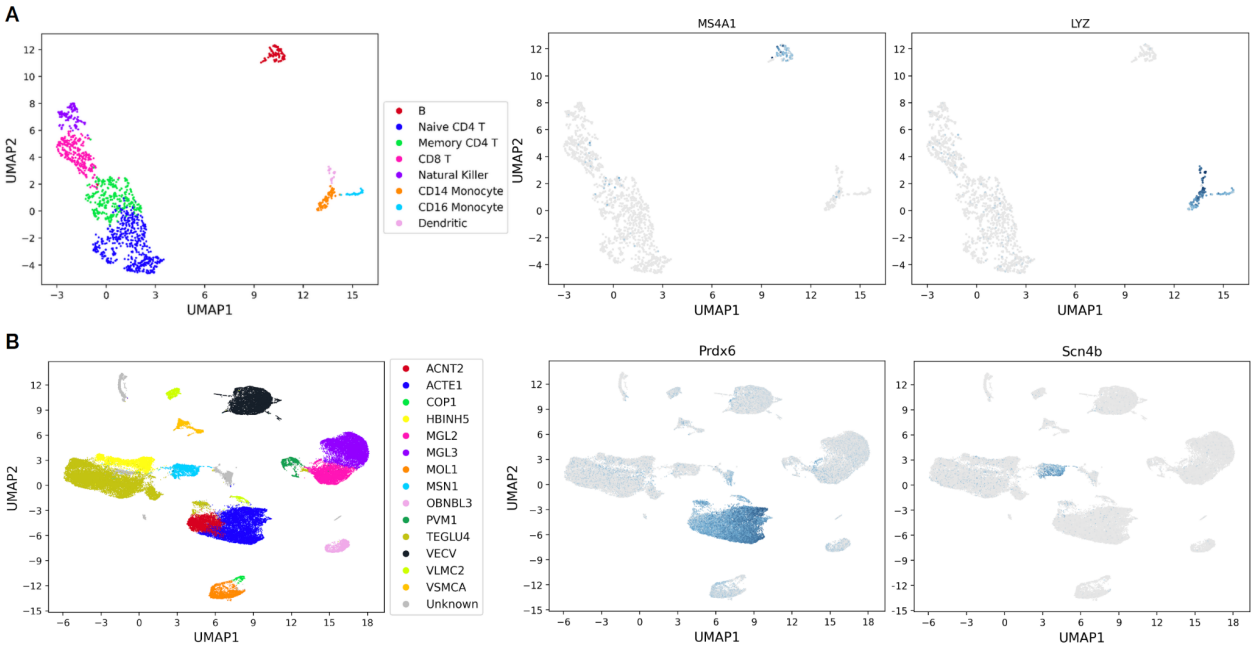


Figure 8: Feature plots from human PBMC and mouse brain samples. A: PBMC annotation and expression maps for MS4A1 and LYZ genes. B: Mouse brain annotation and expression maps for Prdx6 and Scn4b genes.

Input Arguments

The following arguments have the same functionality as in `full` or `cells` modes described above: `--previous`, `--skip-version-check`, `--verbosity`, `--dpi`, `--save-svg`.

`--genes` (string)

A list of genes to create plots for. Should be a comma-separated list (no spaces), e.g., `--genes MALAT,MS4A1,CD14`

`--genes_file` (string)

A list of genes to create plots for. Should be a text file with one gene per row.

`--min-value`, `--max-value` (float)

Minimum and maximum values for plotting. Values outside this range will be assigned the colors at the edge of the colormap. If either is not specified, it would be set automatically to accommodate the expression values for each gene.

`--colormap` (string, default: gray-to-blue)

Colormap to use for plotting. See appendix 1 for available colormaps.

`--colorbar`

Include a colorbar next to the plot.

Generating Cell Type Annotation References: PIPseeker buildannotref

Fluent BioSciences offers downloadable annotation references for several sample types, such as human PBMC and mouse brain. Some users, however, may need annotations for alternative tissues or cell types and may want to create their own reference. PIPseeker buildannotref uses the clustering results from one or more processed samples to create cell type annotation references that can be used to identify cells in future samples of the same type.

`pipseeker buildannotref` takes as input the path to one or more previously generated PIPseeker results, as specified by the `--previous`, `--cell-calling-mode` and `--clustering-mode` arguments. It uses a CSV file with cell types manually assigned to clusters , as specified by `--clusters-file`, whereby it creates a specific signature for each cell type and builds an annotation reference. For example:

```
$ <path to PIPseeker>/pipseeker buildannotref --previous
my/analysis/result/dir/sample1,my/analysis/result/dir/sample1
--cell-calling-mode sensitivity_3,sensitivity_4 --clustering-mode graph,k_4
--clusters-file annotated_clusters.csv --output-file annotation_ref.csv
```

The generated annotation reference will include a list of differential expression scores for each cell type. The scores for cell types defined across multiple samples will be averaged between the samples. See Cell Type Annotation section above for more details about the classification algorithm.

Input Arguments

`--previous` (string)

A path to a directory containing the result of one or more previously completed PIPseeker analyses. If more than one previous result is used, this should be a comma-delimited list of paths..

`--cell-calling-mode` (string)

Cell calling mode (sensitivity_<1-5> or force_<N> in case of manual cell calling). Should be a comma-delimited list if multiple samples are used.

`--clustering-mode` (string)

Clustering mode (graph or k_<N> if K-means clustering was previously added). Should be a comma-delimited list if multiple samples are used.

`--clusters-file` (string)

A path to a CSV file with no header and the following columns:

1) Cluster ID, taken from the clustering results for each sample. The cluster numbers are listed in the output files and shown in the sample reports.
2) Cell type chosen for each cluster, e.g., Monocyte, Hepatocyte, CD34+ (May contain spaces).
3) If more than one sample is used, sample index (starting at 1), based on the order in which they are listed in `--previous`. This column can be omitted if only one sample is used.

Here is an example of a clusters file for PBMC annotation from two PBMC results:

```
4,B,1
1,Naive CD4 T,1
4,Naive CD4 T,2
10,CD8 T,1
11,Natural Killer,1
9,CD14 Monocyte,1
10,CD16 Monocyte,1
1,HSPC,2
```

Note that Naive CD4 is defined for both samples, which means the expression scores for this type will be averaged between the samples.

`--top-genes` (integer, default: 100)

The number of top differentially expressed genes to include from each cluster. The final set of genes included in the generated reference is the union of the lists of top expressing genes from each cluster.

`--weights` (list of integers or floats)

Weights to assign to samples if multiple previous results are included. Should be a comma-delimited list of numbers. This applies only to cell types defined in more than one sample, such as Naive CD4 in the example above. If not defined, all samples will be weighted equally..

`--output-file` (string)

Path to output annotation reference file.

Appendix 1: Colormaps

PIPseeker can create feature UMAP plots for genes and antibody tags using a variety of colormaps. The recommended colormaps are shown below:



Using a colormap that begins with white will result in non-expressing cells being invisible against the white background. To make them visible, choose a colormap that begins with gray.

In addition, any of the standard colormaps listed here can be used:
https://matplotlib.org/stable/gallery/color/colormap_reference.html

Document Revision Summary

Doc ID: FB0002787
Revision: 3.1
Revision Date: March 2023

Legal Notices

© 2022 Fluent BioSciences, Inc (Fluent BioSciences). All rights reserved. Duplication and/or reproduction of all or any portion of this document without the express written consent of Fluent BioSciences, is strictly forbidden. Nothing contained herein shall constitute any warranty, express or implied, as to the performance of any products described herein. Any and all warranties applicable to any products are set forth in the applicable terms and conditions of sale accompanying the purchase of such product.

Fluent BioSciences may refer to the products or services offered by other companies by their brand name or company name solely for clarity, and does not claim any rights in those third party marks or names. The use of products described herein is subject to Fluent BioSciences End User License Agreement, available at www.fluentbio.com/legal-notices, or such other terms that have been agreed to in writing between Fluent BioSciences and the user. All products and services described herein are intended FOR RESEARCH USE ONLY and NOT FOR USE IN DIAGNOSTIC PROCEDURES.

Support

Email: support@fluentbio.com



Fluent BioSciences
150 Coolidge Avenue
Watertown, MA 02472